Week 2 — Neural Networks Revision

Topics: Computational Graphs, Chain Rule, Backpropagation, Gradient Flow, Vanishing/Exploding Gradients, ReLU, Initialization, BatchNorm, and Residual Connections.

> **Goal:** Simple interview-ready explanations with the key concepts and equations. Focus on understanding the intuition rather than memorizing complicated derivations.

1. What is a Computational Graph?

A computational graph shows the sequence of computations performed during the forward pass of a neural network. It also helps us understand how gradients flow backward during backpropagation.

```
x → z1 → h1 → z2 → y_hat → Loss
```

Simple idea: Forward pass goes from input to loss. Backward pass starts at the loss and moves backward through the same graph to calculate gradients.

2. What is the Chain Rule?

The chain rule is used to calculate how the loss changes with respect to parameters by multiplying local derivatives along the computational path.

```
dL/dw = (dL/dy) × (dy/dz) × (dz/dw)
```

In a deep network, we repeatedly multiply partial derivatives from later layers to earlier layers.

3. Explain Backpropagation

Backpropagation is the algorithm used to efficiently calculate gradients of the loss with respect to the weights and biases of a neural network.

Step 1: Perform a forward pass and calculate the prediction.

Step 2: Calculate the loss using the prediction and true target.

Step 3: Start at the loss and move backward through the network.

Step 4: Use the chain rule to calculate gradients for each parameter.

Step 5: Pass those gradients to an optimizer, which updates the weights.

```
θ_new = θ_old − α × ∂L/∂θ
```

Important: Backpropagation calculates gradients. The optimizer (SGD, Adam, etc.) uses those gradients to update parameters.

4. Backpropagation vs Gradient Descent

Backpropagation: Calculates the gradients of the loss with respect to model parameters.

Gradient descent: Uses those gradients to update the parameters.

```
θ_(t+1) = θ_t − α ∇_θ L
```

They are commonly used together, but backpropagation is not limited to vanilla gradient descent. Its gradients can be used by Adam, RMSProp, SGD with momentum, and other optimizers.

5. Why Do Gradients Vanish?

Vanishing gradients occur when gradients become extremely small as they move backward through a deep network.

Backpropagation uses the chain rule, so many derivatives are multiplied together. If the derivatives are consistently smaller than 1, the gradient can shrink exponentially.

```
0.25 × 0.25 × 0.25 × 0.25 → very small
```

For sigmoid, the maximum derivative is only 0.25. In its saturation regions, its derivative approaches zero. Tanh can also have derivatives close to zero in its saturated regions.

If the gradient becomes exactly zero, the corresponding parameter receives no gradient-based update. If it is merely very small, the parameter update becomes extremely small and learning becomes very slow.

6. Why Do Gradients Explode?

Exploding gradients occur when gradients become extremely large as they propagate backward.

Because derivatives are multiplied through many layers, if the relevant factors are consistently greater than 1, the gradient can grow exponentially.

```
10 × 10 × 10 × 10 → very large
```

This can lead to huge weight updates, unstable training, and potentially numerical overflow or NaN values.

Poor initialization with excessively large weights can contribute because weights appear in the derivatives used during backpropagation.

7. How Can We Reduce Vanishing Gradients?

1. Use ReLU-like activations: ReLU has derivative 1 in its active region, so it does not shrink gradients through the activation function there. Leaky ReLU and PReLU provide a small nonzero derivative in the negative region.

2. Proper initialization: Xavier and He initialization help keep activation and gradient variances at reasonable levels.

3. Batch Normalization: Helps keep activations at a controlled scale and can make optimization and gradient flow more stable.

4. Residual connections: Provide a direct path for gradients through very deep networks.

8. How Can We Reduce Exploding Gradients?

1. Proper initialization: Xavier or He initialization helps avoid excessively large activation and gradient scales.

2. Batch Normalization: Helps control activation scale and improve training stability.

3. Residual connections: Improve gradient flow and make deep networks easier to optimize.

4. Gradient clipping: Directly limits very large gradients before the parameter update.

```
If ||g|| > τ, rescale g so that ||g|| = τ
```

Key interview answer: If asked for the most direct technique for exploding gradients, say **gradient clipping**.

9. Why Is ReLU Less Prone to Vanishing Gradients Than Sigmoid?

Sigmoid has a maximum derivative of 0.25 and approaches zero in saturated regions. Repeated multiplication of values below 1 can make gradients extremely small.

$$0.25 \times 0.25 \times 0.25 \times \ldots \rightarrow 0$$

ReLU has derivative 1 in its active region, so multiplying by the activation derivative does not shrink the gradient there.

$$\text{ReLU}'(x) = 1 \text{ for } x > 0$$

However, ReLU still has a zero-gradient region for negative inputs, which leads to the dying ReLU problem.

10. What Is the Dying ReLU Problem?

For standard ReLU, the derivative is zero when the input is negative. If a neuron consistently receives negative inputs, it can stop receiving useful gradient updates and effectively become inactive.

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}'(x) = 0 \text{ for } x < 0$$

Leaky ReLU solves this by using a small nonzero slope in the negative region.

$$\text{Leaky ReLU}(x) = x, \; x > 0$$
$$\text{Leaky ReLU}(x) = \alpha x, \; x < 0$$

Therefore, the derivative is α instead of zero when x is negative, allowing the neuron to continue receiving updates.

11. Can ReLU Still Have Vanishing Gradients?

Yes. ReLU is less prone to vanishing gradients, but it does not completely eliminate them.

Reason 1 — Inactive region: If ReLU receives negative inputs, its derivative is zero.

Reason 2 — Weight matrices: Even when ReLU's derivative is 1, gradients are also multiplied by weight matrices during backpropagation. If these weights are too small, gradients can still shrink.

$$\partial L/\partial a^{(l-1)} = (W^{(l)})^T \times \partial L/\partial z^{(l)}$$

Key idea: ReLU prevents the activation derivative from shrinking gradients in its active region, but it cannot prevent the weight matrices from shrinking gradients.

12. What Is Xavier Initialization?

Xavier (Glorot) initialization aims to keep activation and gradient variance approximately stable across layers.

For a layer with n_in inputs:

$$\text{Var}(z) \approx n_{in} \times \text{Var}(W) \times \text{Var}(x)$$

If Var(W) is approximately 1/n_in, the forward activation variance can remain stable. Backward propagation gives a related requirement involving n_out. Xavier balances the forward and backward requirements.

```
Var(W) = 2 / (n_in + n_out)
```

This is particularly useful for sigmoid and tanh networks. It reduces the chance that activations become extremely large and enter saturated regions where activation derivatives become very small.

Important: Xavier does not guarantee no vanishing gradients. It simply helps maintain healthier activation and gradient scales.

13. What Is He Initialization?

He initialization is designed primarily for ReLU and ReLU-like activation functions.

With a roughly zero-centered symmetric input distribution, ReLU sets approximately half of the inputs to zero. This reduces the signal passing through the activation.

He initialization compensates for this by using a larger weight variance.

```
Var(W) = 2 / n_in
```

Simple mental model: ReLU removes roughly half the signal → He initialization increases the initial variance to compensate.

Comparison:

```
Xavier: Var(W) ≈ 2 / (n_in + n_out)
He: Var(W) ≈ 2 / n_in
```

He is generally preferred for ReLU networks because its initialization accounts for the effect of ReLU on the forward and backward signal.

14. What Is Batch Normalization?

Batch Normalization normalizes activations using the mean and variance of a mini-batch, then applies learnable scale and shift parameters.

```
x_hat = (x − μ_B) / sqrt(σ_B² + ε)
y = γx_hat + β
```

γ (gamma) controls the scale and β (beta) controls the shift. These are learnable parameters, so the network can choose a useful final scale and mean instead of being forced to keep every activation at mean 0 and variance 1.

Why does it help? It keeps activations at a more controlled scale, which can reduce the chance of saturation in sigmoid/tanh and improve numerical and optimization stability.

It can also help with gradient flow because it reduces extreme activation scales, but it does not guarantee that vanishing or exploding gradients will never happen.

15. How Do Residual Connections Help Gradient Flow?

A normal block learns:

```
y = F(x)
```

A residual block instead learns:

```
y = F(x) + x
```

The x is the shortcut or skip connection.

During backpropagation:

```
∂y/∂x = ∂F(x)/∂x + 1
```

The +1 comes from the identity shortcut. This gives the gradient a direct path that does not have to pass entirely through every transformation inside F(x).

For example, if the residual branch has a gradient of:

```
0.5 × 0.5 × 0.5 = 0.125
```

then the residual block has:

```
0.125 + 1 = 1.125
```

The important point is that the +1 is not simply added at the end of one gradient path. The shortcut creates a separate gradient path, and its contribution is added to the gradient from the residual branch.

With many residual blocks, gradients have multiple paths through the network, including shorter paths that skip transformations. This makes very deep networks much easier to train and reduces the risk of vanishing gradients.

Quick Revision: The Big Picture

Computational graph: Shows the forward computations and helps visualize backward gradient flow.

Chain rule: Multiplies local derivatives to calculate gradients.

Backpropagation: Efficiently calculates gradients through the network.

Gradient descent/optimizer: Uses gradients to update parameters.

Vanishing gradient: Repeated multiplication by small derivatives/weights makes gradients approach zero.

Exploding gradient: Repeated multiplication by large derivatives/weights makes gradients grow extremely large.

ReLU: Derivative is 1 in active region, reducing one major source of vanishing gradients.

Leaky ReLU/PReLU: Give negative inputs a nonzero derivative to reduce dying ReLU.

Xavier: Designed to keep forward and backward variance stable; commonly used with tanh/sigmoid.

He: Accounts for ReLU's zeroing of roughly half the signal; commonly used with ReLU.

BatchNorm: Controls activation scale using normalization plus learnable γ and β.

Residual connection: Adds a shortcut path so gradients can flow directly through the network.

Final Interview Challenge

Question: Walk me through backpropagation and explain how vanishing and exploding gradients can occur in a deep neural network. Then explain how you would mitigate them.

Simple answer: During the forward pass, the network computes a prediction and loss. During backpropagation, the chain rule is used to calculate gradients from the loss back through each layer. In a deep network, many derivatives are multiplied together. If these values are consistently below 1, gradients can vanish; if they are consistently above 1, gradients can explode. We can mitigate these problems using suitable activation functions, Xavier or He initialization, BatchNorm, residual connections, and gradient clipping for exploding gradients.

Must-Remember Equations

Chain rule: dL/dw = (dL/dy)(dy/dz)(dz/dw)

Gradient update: θ_new = θ_old − α∇_θL

Sigmoid maximum derivative: 0.25

ReLU derivative: 1 for x > 0, 0 for x < 0

Leaky ReLU derivative: 1 for x > 0, α for x < 0

Xavier variance: Var(W) = 2 / (n_in + n_out)

He variance: Var(W) = 2 / n_in

BatchNorm: x_hat = (x − μ_B) / sqrt(σ_B² + ε)

BatchNorm affine transform: y = γx_hat + β

Residual block: y = F(x) + x

Residual gradient: ∂y/∂x = ∂F(x)/∂x + 1